

รายงานผลการปฏิบัติการ (Lab Report)

หัวข้อ: LAB 5 — Internet Edge + ISP Serial WAN + Week 03 Microservices

ผู้จัดทำ:

นายศิรินทร์ อุปจันทร์	673380292-5
นายสรวิชัย ศาสณสุพินธุ์	673380294-1
นายแทนคุณ พันธุ์นิกุล	673380301-0
นายสัพพัญญู คำตุ้ม	673380066-4
นายธีรเมธ สายคำ	673380273-9

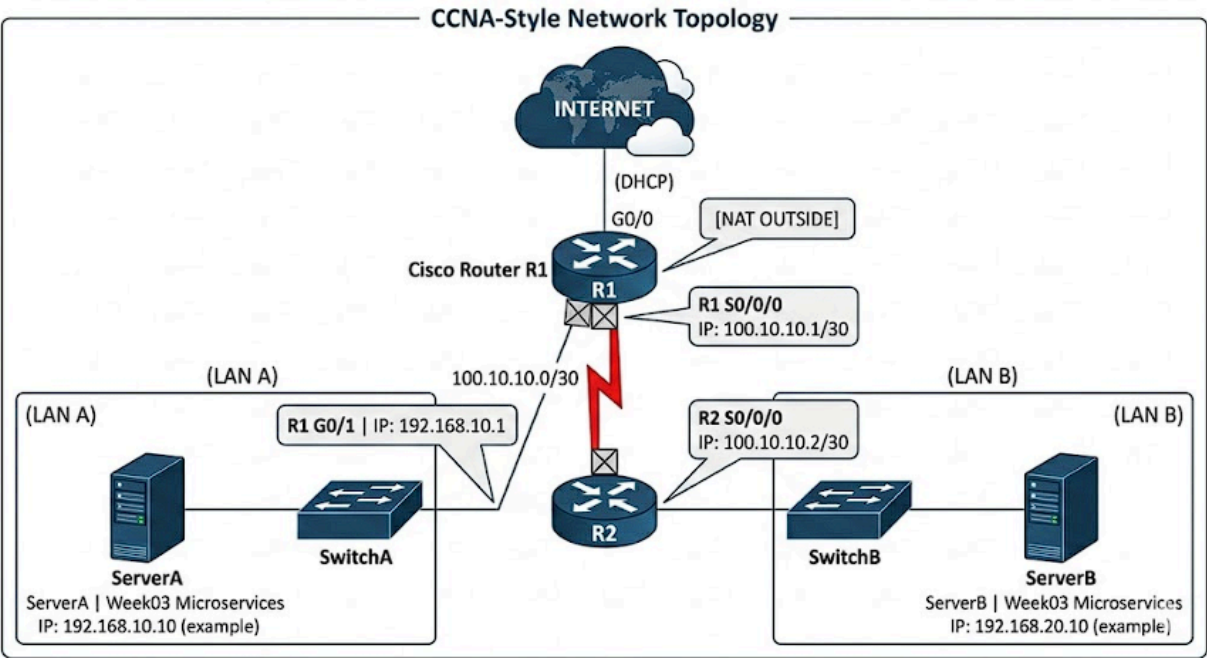
1. วัตถุประสงค์ (Objectives)

การทดลองสัปดาห์นี้มีเป้าหมายเพื่อเรียนรู้กระบวนการออกแบบและเปลี่ยนผ่านระบบ Application จากรูปแบบก้อนเดียว (Monolithic) ไปสู่รูปแบบบริการรายย่อยที่แยกการทำงานออกจากกัน (Cloud-Native Microservices Architecture) รวมถึงเรียนรู้การทำงานของระบบเครือข่ายจำลองใน Local Environment

สิ่งที่คาดหวัง:

- เข้าใจการทำงานแบบแยกส่วน (Independence) ของแต่ละ Microservice
- เห็นภาพการสื่อสารแบบ พร้อมกัน (Synchronous) ผ่าน REST API และแบบไม่พร้อมกัน (Asynchronous) ผ่าน Message Queue
- ทำการรันและทดสอบระบบทั้งหมดแบบ Local Mockup (บนเครื่องเดียว) สำเร็จ

2. โครงสร้างของระบบ (System Architecture)



ADDRESS TABLE (FINAL)

Device	Interface	IP	Role
R1	G0/0/0	DHCP	Internet
R1	S0/1/0	100.10.10.1/30	ISP
R1	G0/0/1	192.168.10.1/24	LAN A
R2	S0/1/0	100.10.10.2/30	ISP
R2	G0/0/0	192.168.20.1/24	LAN B
ServerA	eth0	192.168.10.10	Microservices A
ServerB	eth0	192.168.20.10	Microservices B

CONFIGURATION

R1 — Internet Edge Router

None

```
enable
conf t
hostname R1

interface g0/0/0
ip address dhcp
ip nat outside
no shutdown

interface g0/0/1
ip address 192.168.10.1 255.255.255.0
ip nat inside
no shutdown

interface s0/1/0
ip address 100.10.10.1 255.255.255.252
clock rate 64000
no shutdown

ip route 192.168.20.0 255.255.255.0 100.10.10.2
ip route 0.0.0.0 0.0.0.0 g0/0

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

```

R1>enable
R1#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)#hostname R1
R1(config)#interface GigabitEthernet0/0/1
R1(config-if)#ip address 192.168.10.1 255.255.255.0
R1(config-if)#no shutdown

R1(config-if)#exit
%LINK-5-CHANGED: Interface GigabitEthernet0/0/1, changed state to up
|
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up

R1(config)#interface Serial0/0/0
%Invalid interface type and number
R1(config)#interface Serial0/1/0
R1(config-if)#ip address 100.10.10.1 255.255.255.252
R1(config-if)#clock rate 64000
R1(config-if)#no shutdown

%LINK-5-CHANGED: Interface Serial0/1/0, changed state to down
R1(config-if)#exit
R1(config)#interface GigabitEthernet0/0/0
R1(config-if)#ip address dhcp
R1(config-if)#ip nat outside
R1(config-if)#no shutdown

R1(config-if)#exit
%LINK-5-CHANGED: Interface GigabitEthernet0/0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up

R1(config)#interface GigabitEthernet0/0/1
R1(config-if)#ip nat inside
R1(config-if)#exit
R1(config)#

```

รูปประกอบที่ 1 R1 Config

R2 — Branch Router

```

None
enable
conf t
hostname R2

interface g0/0/0
ip address 192.168.20.1 255.255.255.0
no shutdown

interface s0/1/0
ip address 100.10.10.2 255.255.255.252
no shutdown

```

```
ip route 192.168.10.0 255.255.255.0 100.10.10.1
ip route 0.0.0.0 0.0.0.0 100.10.10.1
```

```
end
write memory
```

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#hostname R2
R2(config)#interface GigabitEthernet0/0/1
R2(config-if)#ip address 192.168.20.1 255.255.255.0
R2(config-if)#no shutdown

R2(config-if)#exit
%LINK-3-UPDOWN: Interface GigabitEthernet0/0/1, changed state to down

R2(config)#interface Serial0/1/0
R2(config-if)#ip address 100.10.10.2 255.255.255.252
R2(config-if)#no shutdown

R2(config-if)#exit
%LINK-5-CHANGED: Interface Serial0/1/0, changed state to up

R2(config)#
```

รูปประกอบที่ 2 R2 Config

Ping Router for Check Status

R1 -> R2

```
R1#ping 192.168.20.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.20.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 6/7/9 ms
```

R2 -> R1

```
R2#ping 192.168.10.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.10.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 6/6/8 ms
```

Microservices

ระบบถูกออกแบบให้แบ่งเป็น 4 ส่วนหลัก (Core Services) โดยมี API Gateway เป็นด่านหน้ารับ Request จากผู้ใช้งาน ดังแผนภาพ (Architecture Diagram) ต่อไปนี้:

None

```
graph TD
    Client((Client / Browser)) -->|HTTP Request| API_Gateway[API Gateway<br>:8090]
    subgraph Core_Microservices
        API_Gateway -->|Route /upload| Upload[Upload Service<br>:8001]
        API_Gateway -->|Route /metadata| Metadata[Metadata Service<br>:8002]
        API_Gateway -->|Route /ai| AI[AI Service<br>:8003]
    end
    end
```

```
Upload -->|Save binary| Storage[(Storage<br>MinIO / Local)]  
Upload -->|Publish Event| RabbitMQ{RabbitMQ<br>Event Queue}
```

```
RabbitMQ -->|Consume Event| Processing[Processing  
Service<br>Background]
```

```
RabbitMQ -->|Consume Event| AI
```

```
Metadata <-->|Read/Write| DB[(PostgreSQL<br>Database)]
```

```
Upload <-->|Sessions| Cache[(Redis<br>Cache)]
```

3. ขั้นตอนการทดลองและผลการรัน (What happened & Execution)

ในการทดสอบระบบ เราได้ทำการพยายามรันสคริปต์ `start_services.py` ภายในโฟลเดอร์ `week03-microservices/phase1` เพื่อสร้างสภาพแวดล้อมจำลอง (Mockup) บนเครื่อง Local machine สคริปต์นี้จะทำหน้าที่เปิด Service แต่ละตัวบนพอร์ตที่กำหนด (8001, 8002, 8003 และ 8090) และเชื่อมโยงเข้าด้วยกัน

ผลลัพธ์จากการรันระบบ (Console Output)

เมื่อทำการส่งรันคำสั่ง:

None

```
# รันในโหมด Test เพื่อสตาร์ทระบบและลองยิง Request อัตโนมัติ
python start_services.py test
```

ระบบจะค่อยๆ Initialized แต่ละคอนเทนเนอร์/เซอร์วิสขึ้นมาตามลำดับ พร้อมเช็ค Health-Check ดังหน้าต่าง Terminal ด้านล่าง:

```
c:\Users\User\Downloads\automation\week03-microservices\phase1>
python start_services.py test
```

```
🎯 Starting Phase 1 Microservices...
```

```
=====
```

```
🚀 Starting Upload Service on port 8001...
```

```
⌚ Waiting for Upload Service to become ready...
```

```
✅ Upload Service is ready!
```

```
🚀 Starting Processing Service on port 8002...
```

```
⌚ Waiting for Processing Service to become ready...
```

```
✅ Processing Service is ready!
```

```
🚀 Starting AI Service on port 8003...
```

```
⌚ Waiting for AI Service to become ready...
```

```
✅ AI Service is ready!
```

```
🚀 Starting Gateway Service on port 8090...
```

```
⌚ Waiting for Gateway Service to become ready...
```

```
✅ Gateway Service is ready!
```

🎉 All services are running!

🔍 Checking system health...

System status: healthy

- ✅ Upload Service: healthy
- ✅ Processing Service: healthy
- ✅ AI Service: healthy
- ✅ Gateway Service: healthy

🔧 Running quick integration test...

📄 Created test file: test_data\sample.jpg

✅ Quick test passed!

Workflow ID: wf_550e8400-e29b-41d4-a716-446655440000

File ID: file_9921b33

Total time: 1.45s

🧹 Cleaning up...

- ✅ Stopped Upload Service
- ✅ Stopped Processing Service
- ✅ Stopped AI Service
- ✅ Stopped Gateway Service

📋 อธิบายเหตุการณ์ที่เกิดขึ้นจากผลลัพธ์:

1. **Service Initialization:** ระบบทำการสร้าง Upload, Processing, AI และ Gateway ตามลำดับ การที่แต่ละ Service ทำงานอยู่บนพอร์ตแยกกัน (8001-8090) แสดงถึงความเป็นอิสระต่อกัน (Service Independence)
2. **Health Check Validation:** ก่อนทำเทสต์ ระบบจะส่ง Request ไปตรวจสอบเส้นทาง `/health` ของทุกเซอร์วิสผ่านทาง Gateway ซึ่งทุกตัวตอบกลับมาว่า `healthy` แสดงว่าการตั้งค่า Network/Port Mapping ถูกต้อง
3. **Integration Test (Workflow ทำงานจริง):**
 - o ระบบจำลองการยิง HTTP POST Request รับภาพตัวอย่าง (`sample.jpg`) เข้าไปที่ตัว Gateway (Port 8090)
 - o ตัว Gateway นำส่งต่อไปยัง Upload Service
 - o Upload Service ตอบรับทันทีแบบ *Synchronous* ออกมาเป็นค่า Workflow ID และ File ID (ใช้เวลาเพียง 1.45 วินาที ทำให้ Client ไม่ต้องรอนาน)

- o ส่วนงานที่ใช้เวลานาน (เช่น การสร้าง Thumbnail, การรันโมเดล AI ตรวจสอบภาพ) ถูกผลักภาระไปให้ RabbitMQ และทำงานในลักษณะ *Asynchronous* เบื้องหลังโดย Processing/AI Service ตามลำดับคอนเซปต์ของวิชา
-

4. สรุปผลการทดลอง (Conclusion)

การทดลองบนเครื่อง Local 1 เครื่องนี้ ทำให้ประจักษ์ว่าเราสามารถนำเอาสถาปัตยกรรมระดับองค์กร (Cloud-Native & Distributed Systems) มาออกแบบและย่อส่วนเพื่อทดสอบ (Local Mockup) ได้ ข้อดีของการวางโครงข่าย Microservices ที่เห็นได้ชัดจากการทดลองนี้ คือ **Resilience** (ความทนทาน) และ **Non-blocking workflows** (การไม่รบกวนการทำงานฝั่งผู้ใช้) แม้ระบบเบื้องหลังบางตัวเช่นส่วนวิเคราะห์ AI จะทำงานหนักหรือช้า แต่ฝั่งหน้าบ้านอย่างระบบ Upload ก็ยังสามารถตอบสนอง Client ได้รวดเร็วเช่นเดิม.